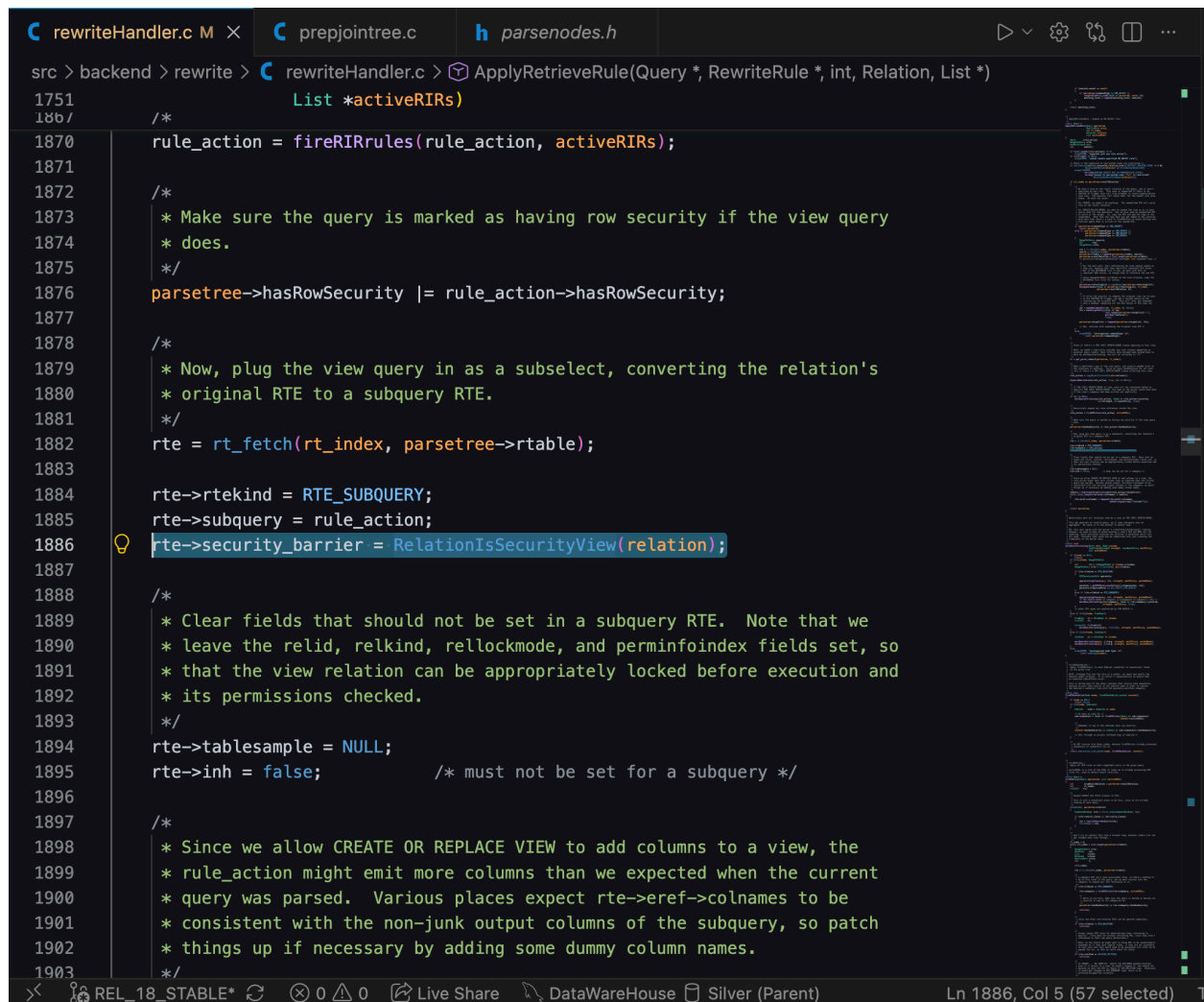


Flag Behaviour:

1. security_barrier:

<https://www.postgresql.org/docs/current/rules-privileges.html>

⇒ In the views we have this flag security_barrier but RLS doesn't have this flag.



```
src > backend > rewrite > C rewriteHandler.c > ApplyRetrieveRule(Query *, RewriteRule *, int, Relation, List *)
1751      List *activeRIRs)
1867      /*
1870      rule_action = fireRIRrules(rule_action, activeRIRs);
1871
1872      /*
1873      * Make sure the query is marked as having row security if the view query
1874      * does.
1875      */
1876      parsetree->hasRowSecurity |= rule_action->hasRowSecurity;
1877
1878      /*
1879      * Now, plug the view query in as a subselect, converting the relation's
1880      * original RTE to a subquery RTE.
1881      */
1882      rte = rt_fetch(rt_index, parsetree->rtable);
1883
1884      rte->rtekind = RTE_SUBQUERY;
1885      rte->subquery = rule_action;
1886      rte->security_barrier = RelationIsSecurityView(relation);
1887
1888      /*
1889      * Clear fields that should not be set in a subquery RTE. Note that we
1890      * leave the relid, relkind, rellockmode, and perminfoindex fields set, so
1891      * that the view relation can be appropriately locked before execution and
1892      * its permissions checked.
1893      */
1894      rte->tablesample = NULL;
1895      rte->inh = false;      /* must not be set for a subquery */
1896
1897      /*
1898      * Since we allow CREATE OR REPLACE VIEW to add columns to a view, the
1899      * rule_action might emit more columns than we expected when the current
1900      * query was parsed. Various places expect rte->eref->colnames to be
1901      * consistent with the non-junk output columns of the subquery, so patch
1902      * things up if necessary by adding some dummy column names.
1903      */
```

1. The rewriter scans range table entries (RTEs) in the query.
2. For a relation RTE, it checks if it has an `ON SELECT` rule (which defines a view).

3. If it finds one, it expands the view using `ApplyRetrieveRule()`, changing the `RangeTblEntry` from a table reference (`RTE_RELATION`) to a subquery (`RTE_SUBQUERY`).
4. It checks if the view options specify it is a security barrier view using the helper `RelationIsSecurityView(relation)`.
5. It sets `rte->security_barrier = true` on the `RangeTblEntry`.

```
/*
 * Now, plug the view query in as a subselect, converting the relation's
 * original RTE to a subquery RTE.
 */
rte = rt_fetch(rt_index, parsetree->rtable);

rte->rtekind = RTE_SUBQUERY;
rte->subquery = rule_action;
rte->security_barrier = RelationIsSecurityView(relation);
```

```
/*
 * RelationIsSecurityView
 * Returns whether the relation is security view, or not. Note multiple
 * eval of argument!
 */
#define RelationIsSecurityView(relation) \
    (AssertMacro(relation->rd_rel->relkind == RELKIND_VIEW), \
     (relation)->rd_options ? \
     ((ViewOptions *) (relation)->rd_options)->security_barrier : false)
```

```
prepjointree.c ×
src > backend > optimizer > prep > prepjointree.c > ...
1814
1815 /*
1816  * is_simple_subquery
1817  *   Check a subquery in the range table to see if it's simple enough
1818  *   to pull up into the parent query.
1819  *
1820  *   rte is the RTE_SUBQUERY RangeTblEntry that contained the subquery.
1821  *   (Note subquery is not necessarily equal to rte->subquery; it could be a
1822  *   processed copy of that.)
1823  *   lowest_outer_join is the lowest outer join above the subquery, or NULL.
1824  */
1825 static bool
1826 is_simple_subquery(PlannerInfo *root, Query *subquery, RangeTblEntry *rte,
1827                   JoinExpr *lowest_outer_join)
1828 {
1829     /*
1830      * Let's just make sure it's a valid subselect ...
1831      */
1832     if (!IsA(subquery, Query) ||
1833         subquery->commandType != CMD_SELECT)
1834         elog(ERROR, "subquery is bogus");
1835
```

```
    /* Don't pull up if the RTE represents a security-barrier view; we
    * couldn't prevent information leakage once the RTE's Vars are scattered
    * about in the upper query.
    */
    if (rte->security_barrier)
        return false;
```

1. Selectivity Estimation & Stats Lookup

- **File:** `c selffuncs.c` (in `examine_simple_variable`)
- **Usage:** If `rte->security_barrier` is `true`, the planner refuses to look up column statistics in `pg_statistic` for that subquery.
- **Why:** To prevent selectivity estimators from running non-leakproof operator functions on underlying data, which could otherwise leak data through timing or side-channel errors.

2. View Options Definitions

- **File:** `c reloptions.c`
- **Usage:** Defines `security_barrier` as a valid boolean option for relation options (e.g., when creating a view with `CREATE VIEW ... WITH (security_barrier = true)`).

3. Query Node Serialization & Deserialization

- **Files:** `c outfuncs.c` and `c readfuncs.c`
- **Usage:** Used to write out (serialize) and read back (deserialize) the `RangeTblEntry` (RTE) query node objects when sending query trees between processes or debugging them.

4. Explain output (in `pg_overexplain` contrib module)

- **File:** `c pg_overexplain.c`
- **Usage:** Checks `rte->security_barrier` to display the `"Security Barrier"` property as part of the extended `EXPLAIN` output.

Here is the comparative flow of the query tree from rewrite to execution for `security_barrier = true` vs `security_barrier = false`.

Phase 1: Query Rewriting (RIR Rule Expansion)

Step	<code>security_barrier = true</code>	<code>security_barrier = false</code>
RTE Conversion	The rewriter converts the view's table reference (<code>RTE_RELATION</code>) into a subquery (<code>RTE_SUBQUERY</code>).	Same.
Flag Assignment	Sets <code>rte->security_barrier = true</code> .	Sets <code>rte->security_barrier = false</code> .
Qual Handling	Attaches view filters to <code>rte->securityQuals</code> .	Attaches view filters to <code>rte->securityQuals</code> .

Phase 2: Optimization (Planning)

Step	<code>security_barrier = true</code>	<code>security_barrier = false</code>
Subquery Pull-up(<code>prepjointree.c</code>)	Blocked: The planner refuses to pull up the subquery. The view remains a separate, isolated subquery in the jointree.	Allowed: The planner flattens/pulls up the subquery, directly merging its tables and filters into the parent query.
Qual Push-down(<code>allpaths.c</code>)	Restricted: Outer query filters containing "leaky" (non-leakproof) functions are blocked from moving down into the view.	Unrestricted: Outer query filters are pushed down freely to the lowest level (e.g. table scan level).
Selectivity Estimation(<code>selfuncs.c</code>)	Blocked: The estimator cannot dig into the subquery's columns to look up <code>pg_statistic</code> entries (preventing side-channel leaks).	Allowed: The estimator digs deep and reads statistics to get accurate selectivity estimations.

Phase 3: Plan Generation & Tree Structure

Step	<code>security_barrier = true</code>	<code>security_barrier = false</code>
Plan Nodes	A nested plan is generated:1. <code>SubqueryScan</code> (evaluates outer leaky filters).2. <code>Seq Scan / Index Scan</code> (evaluates view's security filters).	A flat plan is generated:1. <code>Seq Scan / Index Scan</code> (evaluates both outer filters and view filters).

Phase 4: Execution Runtime

Step	<code>security_barrier = true</code>	<code>security_barrier = false</code>
Qual Evaluation Order	Guaranteed Order: 1. View's security filters run first at base level.2. Out-of-view rows are discarded.3. Leaky outer filters run <i>only</i> on rows that passed the view's filters.	Optimized Order: The engine orders qualifiers by cost/selectivity. Leaky outer filters might run on base rows <i>before</i> the view's filters discard them.
Side-Effects / Security	Leaky functions never see unauthorized rows. No side-channel information is leaked.	Leaky functions may throw errors or log sensitive data from rows the user is not allowed to see.

For the RLS:

Step 1: Query Rewriting (RIR Rule Phase)

In `rewriteHandler.c` (in `fireRIRrules`), the rewriter fetches RLS policies for relation RTEs and attaches them to `rte->securityQuals`:

```
/* 1. Fetch RLS policies from catalog */
get_row_security_policies(parsetree, rte, rt_index, &securityQuals,
    &withCheckOptions, &hasRowSecurity, &hasSubLinks);
/* 2. Attach RLS policies to the RTE */
if (securityQuals != NIL)
{
    // ...
    rte->securityQuals = securityQuals;
}
```

Step 2: Qualifier Distribution (Planning Phase)

In `initsplan.c` (in `deconstruct_distribute`), RLS qualifications are distributed to the relation's restrictions.

```
/* 1. Check if RLS/security quals are present */
if(root->qual_security_level>0)
```

```
process_security_barrier_qual(s(root, varno, jtitem);
```

Then inside **process_security_barrier_qual(s()**, RLS quals are assigned a low security level starting at **0**:

```
static void
process_security_barrier_qual(s(PlannerInfo *root, int rti, JoinTreeItem *jtitem)
{
    RangeTblEntry *rte = root->simple_rte_array[rti];
    Index          security_level = 0;
    ListCell       *lc;

    foreach(lc, rte->securityQuals)
    {
        List *qualset = (List *) lfirst(lc);

        /* Distribute the RLS qual with low security_level
(0, 1, ...) */
        distribute_qual(s_to_rels(root, qualset,
                                jtitem,
                                NULL,
                                security_level, // <-- RLS
priority level
                                jtitem->qualscope,
                                ...));

        security_level++;
    }
}
```

(Meanwhile, the user's outer query quals are distributed in `deconstruct_distribute()` using `root->qual_security_level`, which is greater than any RLS policy's level).

Step 3: Qualifier Sorting (Plan Generation Phase)

Before creating a physical plan scan node (like `SeqScan`), the planner orders the quals. In **createplan.c** (in `order_qual_clauses`):

```
static List *
order_qual_clauses(PlannerInfo *root, List *clauses)
{
    // ...
    /* 1. Read security levels of each qualification */
    i = 0;
    foreach(lc, clauses)
    {
        Node *clause = (Node *) lfirst(lc);
        // ...
        if (IsA(clause, RestrictInfo))
        {
            RestrictInfo *rinfo = (RestrictInfo *) clause;

            /* Exception: Cheap leakproof quals can run at le
            vel 0 */
            if (rinfo->leakproof && items[i].cost < 10 * cpu_
            operator_cost)
                items[i].security_level = 0;
            else
                items[i].security_level = rinfo->security_lev
            el;
        }
        i++;
    }

    /* 2. Sort the quals using a stable insertion sort */
    for (i = 1; i < nitems; i++)
    {
        QualItem newitem = items[i];
        int j;

        for (j = i; j > 0; j--)
```

```

    {
        QualItem *olditem = &items[j - 1];

        /* Sort by security_level (lowest first), then by
        execution cost */
        if (newitem.security_level > olditem->security_level ||
            (newitem.security_level == olditem->security_level &&
             newitem.cost >= olditem->cost))
            break;
        items[j] = *olditem;
    }
    items[j] = newitem;
}
// ...
}

```

The resulting sorted list places RLS quals at the beginning. This list is passed to `make_seqscan()` or other scan nodes, forcing the executor to evaluate RLS filters first.

the insertion sort places items with the **lowest** `security_level` **at the front of the list**, the RLS policy conditions (which start with `security_level = 0`) get the **highest priority** and are evaluated first.

The only other filters that can share this top priority are user-supplied filters that are **both leakproof and extremely cheap** (which are also temporarily set to `security_level = 0` for performance). Since they are mathematically proven to be leakproof, letting them run early is completely safe.

All other user query filters (which have a higher `security_level` value) are forced to wait at the back of the queue until the RLS policy filters have processed the rows.

code flow for how the planner builds a nested plan for a view with `security_barrier = true` :

Step 1: Dispatch to Subquery Scan Plan Creator

During the final plan-creation phase, **createplan.c** (in `create_plan_recurse`) detects a `SubqueryScanPath` (which exists because the subquery/view was blocked from being pulled up):

```
case T_SubqueryScanPath:
    plan = (Plan *) create_subqueryscan_plan(root, (SubqueryScanPath *) best_path, tlist, scan_clauses);
    break;
```

Step 2: Recursively Plan the View and Wrap It

Inside **create_subqueryscan_plan()**, the planner builds a nested physical layout:

```
static SubqueryScan *
create_subqueryscan_plan(PlannerInfo *root, SubqueryScanPath
*best_path,
                        List *tlist, List *scan_clauses)
{
    SubqueryScan *scan_plan;
    Plan          *subplan;

    /* 1. Recursively generate the plan for the view's interior.
     *    This inner subplan will contain the view's security barrier
     *    quals. */
    subplan = create_plan(rel->subroot, best_path->subpath);

    /* 2. Process outer leaky filters (scan_clauses) */
    scan_clauses = order_qual_clauses(root, scan_clauses);
    scan_clauses = extract_actual_clauses(scan_clauses, false);

    // ... (param replacement etc.)
```

```

/* 3. Build the final wrapper node (SubqueryScan) around
the subplan */
scan_plan = make_subqueryscan(tlist,
                                scan_clauses, // Evaluated
on SubqueryScan
                                scan_relid,
                                subplan);    // Inner plan
evaluating security barrier quals

return scan_plan;
}

```

Resulting Physical Plan Structure

The executor receives the plan structured like this:

```

      [ SubqueryScan ]
Quals: (outer leaky filters)
      |
      v
    [ Seq Scan / Index Scan ]
Quals: (view security barrier filters)

```

code flow for how the planner builds a flat query plan when a view has `security_barrier = false`:

Step 1: Subquery is Pulled Up (Dissolved)

During optimizer preprocessing, **prepjointree.c**

(in `pull_up_subqueries_recurse`) checks the flag. Since `rte->security_barrier` is `false`, it proceeds to pull up the view:

```

if (rte->rtekind == RTE_SUBQUERY &&
    is_simple_subquery(root, rte->subquery, rte, lowest_outer
_join))
{

```

```

/* Pull up the subquery/view */
return pull_up_simple_subquery(root, jtnode, rte, ...);
}

```

Step 2: Merge Range Tables and Qualifications

Inside `pull_up_simple_subquery()`, the view's query block is completely dissolved:

```

/* 1. Replace outer variables referencing the view columns
 *    with the actual column expressions inside the view. */
perform_pullup_replace_vars(root, &rvcontext, containing_appe
ndrel);

/* 2. Merge the view's range table (tables) into the parent q
uery's range table */
CombineRangeTables(&parse->rtable, &parse->rteperminfos, subq
uery->rtable, subquery->rteperminfos);

/* 3. Return the view's inner jointree node to be merged dire
ctly
 *    into the parent jointree. */
return (Node *) subquery->jointree;

```

Step 3: Base Relation Path Generation

Because the view is gone and its underlying tables are now part of the parent query, the planner generates standard scan paths on the base table.

In `allpaths.c` (in `set_plain_rel_pathlist`), paths like `SeqScanPath` or `IndexPath` are generated. Crucially, the base table's `baserestrictinfo` list now contains **both** the view's inner filters and the outer query's filters merged together.

Step 4: Plan Node Generation

During the plan-creation phase, the planner encounters a normal `SeqScanPath` (instead of a `SubqueryScanPath`).

In `createplan.c` (in `create_seqscan_plan`), it simply creates a flat `SeqScan` plan node:

```
static SeqScan *make_seqscan(List *qptlist, List *qpqual,
/* Contains BOTH view filters and outer filters */ Index scan
relid)
{
    SeqScan    *node = makeNode(SeqScan);
    Plan       *plan = &node->scan.plan;

    plan->targetlist = qptlist;
    plan->qual = qpqual;      // Flat list of all filters
    plan->lefttree = NULL;
    plan->righttree = NULL;
    node->scan.scanrelid = scanrelid;

    return node;
}
```

Resulting Physical Plan Structure

The executor receives a single flat scan node:

```
[ Seq Scan / Index Scan ]
Quals: (view filters AND outer filters)
```

`security_barrier = true` (Recursively Plan the View)

1. **Isolation:** The view `dev_employees` is kept as a separate query block (`RTE_SUBQUERY`). The planner cannot merge the filters.
2. **Recursion** (`subplan = create_plan(rel->subroot, best_path->subpath)`):
 - The planner starts planning the main query, but when it encounters the subquery (the view), it **recursively invokes itself** to plan the view first.
 - This sub-planner plans the view's query: `SELECT * FROM employees WHERE department = 'development'`

- It produces an **inner subplan**:

```
[ Seq Scan on employees]
Filter: (department = 'development')
```

3. **Wrapping**: The outer planner takes that completed inner subplan and wraps it inside a **SubqueryScan** node. It attaches the outer query filter (`salary > 100000`) to the outer node:

```
[ SubqueryScan]
Filter: (salary> 100000)
      |
      v
[ Seq Scan on employees] (Pre-planned inner subplan)
Filter: (department = 'development')
```

RLS Policy with UDF (Opaque Interpreter Loop)

When running an RLS policy with `USING (my_security_check(owner))`, the VM executes `EEOP_FUNCEXPR`, but the function pointer `fn_addr` points to the **PL/pgSQL language call handler** (`plpgsql_call_handler`) instead of a simple compiled C function:

```
EEO_CASE(EEOP_FUNCEXPR)
{
    FunctionCallInfo fcinfo = op->d.func.fcinfo_data;
    Datum            d;

    fcinfo->isnull = false;
    /* Invokes the PL/pgSQL engine handler for each row! */
    d = op->d.func.fn_addr(fcinfo);
    *op->resvalue = d;
    *op->resnull  = fcinfo->isnull;
}
```

```

    EEO_NEXT();
}

```

What happens inside `fn_addr(fcinfo)` when PL/pgSQL runs:

1. **Context Initialization:** PL/pgSQL creates a memory context and pushes a new interpreter execution frame (`PLpgSQL_execstate`) onto its stack.
2. **Variable Binding:** Copies `owner` from `fcinfo` arguments into the PL/pgSQL internal variables.
3. **SPI Execution:** Since the policy check queries metadata (e.g., roles or groups), the UDF calls the **Server Programming Interface (SPI)**:
 - SPI prepares a nested command execution state.
 - SPI triggers a **nested, recursive execution of the Postgres planner and executor** to run the lookup query (`SELECT role FROM user_roles WHERE username = $1`).
4. **Data Extraction:** SPI reads the query result, extracts the boolean `true` / `false` , and passes it back to PL/pgSQL.
5. **Context Cleanup:** PL/pgSQL tears down its stack frame and frees variables.

Query 4:

Normal Query:

```

select
    o_orderpriority,
    count(*) as order_count
from
    orders
where
    o_orderdate >= date '1994-01-01'
    and o_orderdate < date '1994-01-01' + interval '3' mo
nth

```

```

        and exists (
            select
                *
            from
                lineitem
            where
                l_orderkey = o_orderkey
                and l_commitdate < l_receiptdate
        )
    group by
        o_orderpriority
    order by
        o_orderpriority;

```

Analysis of Outputs:

1. Views (Without secure_view)

```

SQL Query
  ↓
Parser
  ↓
Analyzer
  ↓
Rewrite
  ↓
View Expansion
  ↓
Planner
  ↓
View Pull-Up / Flattening
  ↓
Predicate Pushdown

```

↓
Join Optimization
↓
Executor

Step 1: Query References a View

Suppose Q4 contains:

```
SELECT ...  
FROM orders_secure  
...
```

In the analyzed query, `orders_secure` appears as:

```
RTE_RELATION  
relkind = VIEW
```

Step 2: Rewrite System Detects It's a View

File: `src/backend/rewrite/rewriteHandler.c`

Functions:

```
QueryRewrite()  
↓  
fireIRIRules()  
↓  
ApplyRetrieveRule()
```

`fireIRIRules()` checks every RTE. When it sees:

```
orders_secure
```

it looks up the `_RETURN` rule stored for the view. Then it replaces:

```
orders_secure
```

with

```
RTE_SUBQUERY
```

containing:

```
SELECT*  
FROM orders  
WHERE EXISTS(...)
```

Step 3: Is security_barrier Checked Here?

Not exactly. The rewrite system always expands the view. The security barrier flag is stored in the RTE.

Look in: `src/backend/rewrite/rewriteHandler.c`

around:

```
ApplyRetrieveRule()
```

You'll find:

```
rte->security_barrier=RelationIsSecurityView(relation);
```

So after expansion:

```
RTE_SUBQUERY  
    security_barrier = false
```

for a normal view. or

```
RTE_SUBQUERY
    security_barrier = true
```

Step 4: Planner Receives Rewritten Query

File: src/backend/optimizer/plan/planner.c

Function:

```
planner()
```

↓

```
subquery_planner()
```

Step 5: Pull-Up / Flattening

File: src/backend/optimizer/prep/prepjointree.c

Functions:

```
pull_up_subqueries()
```

↓

```
is_simple_subquery()
```

This is where the important check happens.

PostgreSQL asks:

```
Can I merge this view into the parent query?
```

One of the checks is:

```
if (rte->security_barrier)
returnfalse;
```

(inside the pull-up logic)

So:

```
security_barrier = false
```

✓ Pull up allowed

```
security_barrier = true
```

✗ Pull up blocked

Step 6: View Gets Merged

After pull-up: Instead of:

```
Parent Query
  ↓
Subquery(View)
  ↓
orders
```

Planner gets:

```
orders
customer
lineitem
```

all inside one query tree.

Now optimizer can:

- convert EXISTS to Semi Join
- push predicates

- reorder joins
- choose better join methods

Step 7: Optimization

Still inside: src/backend/optimizer/prep/prepjointree.c

Functions:

```
pull_up_sublinks()
convert_EXISTS_sublink_to_join()
```

Then later:

```
src/backend/optimizer/path/
```

Functions:

```
make_one_rel()
standard_join_search()
```

generate Hash Join, Hash Semi Join, etc.

```
View?
  ↓
Rewrite expands view
  ↓
Planner checks security_barrier
  ↓
security_barrier = false ?
  YES → merge/pull-up view
        ↓
        optimize whole query together
  NO  → keep subquery boundary
        ↓
        fewer optimizations
```

↓
Executor